VMAS (VMA Services)

Comprehensive Project Documentation & System Architecture

Project:	VMAS Online Ready-Made Food Packs Delivery Platform
Tech Stack:	React (Vite) + Tailwind CSS v4 | Django REST Framework | Supabase (PostgreSQL)
Architecture:	Decoupled Frontend (Vite) & Backend (DRF) Monorepo
Date:	October 18, 2026
Repository:	github.com/dharuntech275/vmas

Hot rotis at your door, ready to eat. Sold exclusively as family-friendly packs.

1. Project Overview

VMAS (VMA Services) is an on-demand food delivery platform inspired by modern ordering apps (Swiggy/Zomato), designed specifically for freshly made, ready-to-eat bread packs rather than full restaurant meals. VMAS sells wholesome staple foods (Chapathi, Wheat Parotta, Plain Naan, Butter Naan, Phulka, and Poori) packaged exclusively in multi-piece family portions (e.g., Pack of 5, Pack of 4).

Core Value Proposition: Customers eliminate tedious dough kneading, rolling, and cooking. Breads are prepared fresh upon order placement and rushed hot to customer doorsteps.

2. Tech Stack & Design System

Component	Technology & Purpose
Frontend Framework	React 19 + Vite (Fast HMR, optimized production bundler)
Styling System	Tailwind CSS v4 (Integrated via @tailwindcss/vite plugin)
Routing & State	React Router v7 + React Context API with localStorage synchronization
Backend Framework	Python 3.14 + Django 6 + Django REST Framework (DRF)
Database	Supabase PostgreSQL direct connection (SSL enforced: sslmode=require) with local SQLite fallback
Production Server	Gunicorn WSGI server + WhiteNoise static asset engine
Brand Identity	Banana-leaf green (#14452F), Turmeric gold (#F2B705), Off-white (#FBFBFA), Google Font *Bricolage Grotesque*, Light & Dark themes

3. Monorepo Folder Structure

```
vmas/
■■■ backend/ # Django REST Framework Backend
■ ■■■ config/ # Project settings, URLs, WSGI, ASGI
■ ■■■ menu/ # Menu items, serializers, seed commands, admin
■ ■■■ orders/ # Orders, items, atomic checkout, tracking
■ ■■■ requirements.txt # Python dependencies
■ ■■■ manage.py
■■■ frontend/ # React + Vite Client Application
■ ■■■ src/
■ ■ ■■■ api/client.js # Generic fetch API client
■ ■ ■■■ components/ # Navbar, MenuCard, CartItem, Footer, StatusSteps
■ ■ ■■■ context/ # CartContext, ThemeContext, ToastContext
■ ■ ■■■ pages/ # Home, Menu, Cart, Checkout, TrackOrder, NotFound
■ ■ ■■■ index.css # Tailwind CSS v4 imports & theme tokens
■ ■■■ vercel.json # Vercel SPA deployment rules
■ ■■■ package.json
■■■ docs/ # Project Documentation & Automated PDF Generator
■ ■■■ CHANGELOG.md # Step-by-step development history
■ ■■■ generate_pdf.py # ReportLab documentation builder
■ ■■■ VMAS_Documentation.pdf # Generated comprehensive PDF
■■■ .gitignore # Excludes .env, venv, node_modules, sqlite3
■■■ README.md # Comprehensive project onboarding guide
```

4. Database Architecture & Models

Model Name	Fields, Types & Constraints
MenuItem (menu app)	• name (CharField 120) • slug (SlugField 150, unique, auto-generated) • description (TextField, optional) • pack_size (CharField 50, e.g. 'Pack of 5') • price (DecimalField 8,2 in INR ■) • image (URLField 500, direct URL only, no uploads) • is_available (BooleanField, default=True) • created_at (DateTimeField auto_now_add)
Order (orders app)	• customer_name (CharField 120) • phone (CharField 15, validated 10-digit Indian mobile) • address (TextField, minimum 8 characters) • payment_method (Choices: COD, UPI) • status (Choices: RECEIVED, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELLED) • total (DecimalField 10,2, backend-calculated) • created_at (DateTimeField auto_now_add)
OrderItem (orders app)	• order (ForeignKey to Order, on_delete=CASCADE, related_name='items') • menu_item (ForeignKey to MenuItem, on_delete=PROTECT) • quantity (PositiveIntegerField, validated 1 to 20) • unit_price (DecimalField 8,2, price snapshot at time of checkout) • subtotal (Computed property: quantity * unit_price)

5. REST API Specifications

Endpoint	Method	Description & Request / Response Schema
/api/menu/	**GET**	Returns active available food packs (is_available=True). **Response (200):** [{"id": 1, "name": "Chapathi", "pack_size": "Pack of 5", "price": "50.00", ...}]
/api/orders/	**POST**	Places a new order atomically. Backend calculates total from DB rates. **Request Body:** {"customer_name": "...", "phone": "9876543210", "address": "...", "payment_method": "COD", "items": [{"menu_item": 1, "quantity": 2}]} **Response (201):** {"id": 1, "status": "RECEIVED", "total": "200.00", "items": [...]}
/api/orders/<id>/?phone=...	**GET**	Tracks live order status. Enforces phone security verification. **Response (200):** Full order details with StatusSteps timeline. **Response (404):** Returned when phone is missing or does not match.

6. Setup, Run & Production Deployment Guide

Local Development Execution:

• Backend: cd vmas/backend && venv\Scripts\activate && python manage.py runserver (port 8000)
• Frontend: cd vmas/frontend && npm run dev (port 5173)
• Automated Test Suite: python manage.py test (8 tests verify calculations & validations)

Production Deployment Checklist:
1. **Backend (Render / Railway):** Set environment variables (SECRET_KEY, DEBUG=False, DB_* for Supabase, ALLOWED_HOSTS, CORS_ALLOWED_ORIGINS). Build Command: pip install -r requirements.txt && python manage.py collectstatic --noinput && python manage.py migrate && python manage.py seed_menu. Start Command: gunicorn config.wsgi:application.
2. **Frontend (Vercel / Netlify):** Set VITE_API_URL=https://your-backend.onrender.com/api. Build Command: npm run build. Output directory: dist.
3. **Static Assets:** WhiteNoise serves compressed assets directly from STATIC_ROOT.
4. **Secrets Protection:** .env files and credentials strictly isolated via .gitignore.

7. Step-by-Step Implementation Changelog

The following log is parsed from CHANGELOG.md detailing milestones from inception through deployment:

Step 1: Backend Setup & Supabase Database Configuration (2026-10-17)

Summary: Initialized Django project `config` in `vmas/backend`. Created virtual environment `venv` and installed dependencies (`django`, `djangorestframework`, `psycopg2-binary`, `django-cors-headers`, `python-dotenv`, `gunicorn`, `reportlab`). Created `menu` and `orders` applications. Configured `settings.py` with `python-dotenv` for loading sensitive secrets, enabled direct connection to Supabase PostgreSQL with SSL (`sslmode=require`), CORS for `http://localhost:5173`, and set `TIME_ZONE = 'Asia/Kolkata'`.

Files: `vmas/backend/requirements.txt`, `vmas/backend/manage.py`, `vmas/backend/config/settings.py`, `vmas/backend/config/urls.py`, `vmas/backend/config/wsgi.py`, `vmas/backend/config/asgi.py`, `vmas/backend/menu/` (app scaffolding), `vmas/backend/orders/` (app scaffolding), `vmas/backend/.env.example`, `vmas/backend/.gitignore`

```
cd vmas/backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python manage.py check
---
```

Step 2: Django Database Models, Admin & Seed Management Command (2026-10-17)

Summary: Defined `MenuItem` model in `menu` app with auto-generating slug, INR decimal price, and pack sizes. Defined `Order` and `OrderItem` models in `orders` app with choices for payment method (`COD`, `UPI`) and order status (`RECEIVED`, `PREPARING`, `OUT_FOR_DELIVERY`, `DELIVERED`, `CANCELLED`). Registered models in Django Admin with `OrderItemInline` and `list_editable = ('status',)`. Created `seed_menu` command to populate 6 initial food packs. Created and executed migrations.

Files: `vmas/backend/menu/models.py`, `vmas/backend/menu/admin.py`, `vmas/backend/orders/models.py`, `vmas/backend/orders/admin.py`, `vmas/backend/menu/management/commands/seed_menu.py`, `vmas/backend/menu/migrations/0001_initial.py`, `vmas/backend/orders/migrations/0001_initial.py`

```
python manage.py makemigrations
python manage.py migrate
python manage.py seed_menu
python manage.py createsuperuser
---
```

Step 3: REST API (Django REST Framework) & Automated Unit Tests (2026-10-17)

Summary: Built DRF endpoints: `GET /api/menu/` for available food packs, `POST /api/orders/` with backend total calculation, price snapshotting in `OrderItem`, and `transaction.atomic()`. Built `GET /api/orders//?phone=...` tracking endpoint enforcing 404 on phone mismatch. Added strict validations (10-digit Indian mobile starting with 6-9, address min 8 chars, quantity 1-20, unavailable item rejection). Wrote 8 comprehensive unit tests in `orders/tests.py` and `menu/tests.py`.

Files: `vmas/backend/menu/serializers.py`, `vmas/backend/menu/views.py`, `vmas/backend/menu/urls.py`, `vmas/backend/orders/serializers.py`, `vmas/backend/orders/views.py`, `vmas/backend/orders/urls.py`, `vmas/backend/config/urls.py`, `vmas/backend/orders/tests.py`, `vmas/backend/menu/tests.py`

```
python manage.py test
python manage.py runserver 127.0.0.1:8000
---
```

Step 4: Frontend Setup (React, Vite, Tailwind CSS v4 & Client Routing) (2026-10-17)

Summary: Scaffolded client in `vmas/frontend` using Vite + React 19. Integrated `@tailwindcss/vite` (Tailwind CSS v4) with custom branding (Banana-leaf green `#14452F`, Turmeric gold `#F2B705`, off-white `#FBFBFA`). Added Google Font `Bricolage Grotesque`. Implemented `CartContext.jsx` with `localStorage` persistence, `ThemeContext.jsx` for persistent dark mode toggling, `api/client.js` fetch wrapper, and initial components (`Navbar`, `MenuCard`, `CartItem`, `Footer`, `StatusSteps`). Set up 5 client routes with `react-router-dom`.

Files: `vmas/frontend/package.json`, `vmas/frontend/vite.config.js`, `vmas/frontend/index.html`, `vmas/frontend/src/index.css`, `vmas/frontend/src/App.jsx`, `vmas/frontend/src/api/client.js`, `vmas/frontend/src/context/CartContext.jsx`, `vmas/frontend/src/context/ThemeContext.jsx`, `vmas/frontend/src/components/Navbar.jsx`, `vmas/frontend/src/components/MenuCard.jsx`, `vmas/frontend/src/components/CartItem.jsx`, `vmas/frontend/src/components/Footer.jsx`, `vmas/frontend/src/components/StatusSteps.jsx`, `vmas/frontend/src/pages/Home.jsx`, `vmas/frontend/src/pages/Menu.jsx`, `vmas/frontend/src/pages/Cart.jsx`, `vmas/frontend/src/pages/Checkout.jsx`, `vmas/frontend/src/pages/TrackOrder.jsx`, `vmas/frontend/.env.example`, `vmas/frontend/.gitignore`

```
cd vmas/frontend
npm install
npm run build
npm run dev
---
```

Step 5: Full Pages Implementation & End-to-End API Integration (2026-10-17)

Summary: Connected all frontend pages to live Django API endpoints. `Home` displays hero banner, "Order now" CTA, live popular packs, and 3-step "How it works" guide. `Menu` fetches `/api/menu/` with loading skeleton, error retry, search bar, and interactive Add / + / - stepper on `MenuCard`. `Cart` provides line-item modifiers, total breakdown, and empty-state "Menu-ku po" button. `Checkout` features customer inputs, frontend validations, field-wise backend error rendering, atomic order creation, and cart reset. `TrackOrder` provides live timeline, phone verification, and 15-second automatic refresh. Successfully executed end-to-end full order test.

Files: `vmas/frontend/src/pages/Home.jsx`, `vmas/frontend/src/pages/Menu.jsx`, `vmas/frontend/src/pages/Cart.jsx`, `vmas/frontend/src/pages/Checkout.jsx`, `vmas/frontend/src/pages/TrackOrder.jsx`, `vmas/frontend/src/components/MenuCard.jsx`, `vmas/backend/config/settings.py`, `vmas/backend/menu/management/commands/seed_menu.py`

```
python manage.py runserver 127.0.0.1:8000
npm run dev
---
```

Step 6: Django Admin Polish, Bulk Actions, Toasts, 404 Page & README (2026-10-18)

Summary: Enhanced Django Admin with order bulk actions ("Mark as Preparing", "Mark as Out for delivery", "Mark as Delivered"), search by customer name, phone, and ID, filters by status, payment method, and date hierarchy. Configured `MenuItemAdmin` with URL-only image fieldset (direct uploads disabled) and inline `is_available` toggles. Built frontend `NotFound.jsx` (404 page), interactive toast notification system (`ToastContext.jsx`) for Add to Cart and Order Placed events, dynamic page titles (`useDocumentTitle` hook), and high-contrast `:focus-visible` accessibility styles. Authored comprehensive root `README.md`.

Files: `vmas/backend/orders/admin.py`, `vmas/backend/menu/admin.py`, `vmas/frontend/src/context/ToastContext.jsx`, `vmas/frontend/src/hooks/useDocumentTitle.js`, `vmas/frontend/src/pages/NotFound.jsx`, `vmas/frontend/src/App.jsx`, `vmas/frontend/src/components/MenuCard.jsx`, `vmas/frontend/src/pages/Checkout.jsx`, `vmas/frontend/src/pages/Home.jsx`, `vmas/frontend/src/pages/Menu.jsx`, `vmas/frontend/src/pages/Cart.jsx`, `vmas/frontend/src/pages/TrackOrder.jsx`, `vmas/frontend/src/index.css`, `README.md`

```
python manage.py check
npm run build
---
```

Step 7: Deployment Preparation, Comprehensive PDF Generator & GitHub Push (2026-10-18)

Summary: Integrated `whitenoise` for serving static assets in production with `STATIC_ROOT` and `CompressedManifestStaticFilesStorage`. Configured environment variables for production `DEBUG=False`, `ALLOWED_HOSTS`, `CORS_ALLOWED_ORIGINS`, and `CSRF_TRUSTED_ORIGINS`. Added Vercel (`vercel.json`) and Netlify (`public/_redirects`) SPA routing configuration. Created `vmas/docs/CHANGELOG.md` and authored comprehensive `vmas/docs/generate_pdf.py` script producing `vmas/docs/VMAS_Documentation.pdf` with cover page, project overview, tech stack, folder structure, database schema, API documentation, deployment checklists, and changelog. Prepared monorepo Git structure on `main` branch ready for GitHub push to `dharuntech275/vmas`.

Files: `vmas/backend/requirements.txt`, `vmas/backend/config/settings.py`, `vmas/backend/.env.example`, `vmas/frontend/vercel.json`, `vmas/frontend/public/_redirects`, `vmas/frontend/.env.example`, `vmas/docs/CHANGELOG.md`, `vmas/docs/generate_pdf.py`, `vmas/docs/VMAS_Documentation.pdf`, `vmas/.gitignore`, `vmas/README.md`

```
python vmas/docs/generate_pdf.py
git init
git remote add origin https://github.com/dharuntech275/vmas.git
```